Platepal: Project 2 Proposal

1. Application Purpose and Description

PlatePal is an AI-powered web application designed to help users better understand the nutritional content of their meals. Building upon the React single-page application developed in Project 1, Project 2 will transform the existing demonstration into a functional application that integrates Google Gemini, Google Agent Development Kit (ADK), Firebase Authentication, Google Firestore, and a Node.js/Express backend.

Users will be able to select meal photographs from an authenticated external photo service and optionally record spoken descriptions containing additional information about ingredients, portion sizes, or food preparation. Google Gemini will analyze the photographs and audio to identify food items and generate estimated nutritional information, including calories and macronutrients.

PlatePal will also feature an intelligent meal-analysis agent developed using Google ADK. This agent will coordinate the AI analysis process and generate recommendations based on the user's selected companion style: Casual, Strict, or Athlete.

Analysis results will be presented through an interactive React interface, allowing users to review identified foods, nutritional estimates, and personalized recommendations. Authenticated users will also be able to save their results to Google Firestore and retrieve previous meal analyses.

The goal of PlatePal is to provide users with an accessible, engaging way to explore their eating habits and receive helpful AI-generated nutritional insights. Nutritional values will be presented as estimates rather than medically verified measurements.

User Authentication

PlatePal will use Firebase Authentication to provide secure user registration and sign-in functionality. The existing LoginForm.jsx and CreateAccountForm.jsx components from Project 1 will be reused and enhanced to support authenticated user accounts.

Users will register using an email address and password, with the option to maintain a display name and username in their profile. Firebase Authentication will manage account credentials and authentication sessions. The React frontend will obtain authentication tokens, which the Node.js/Express backend will verify before processing protected requests.

Authenticated users will be able to access their saved meal analyses, preferences, and meal histories stored in Google Firestore. Each user's records will be associated with their unique authentication identifier to prevent unauthorized access.

PlatePal will also require users to authorize access to an approved external photo service. This authorization process will be separate from PlatePal account authentication and will use the photo provider's supported authentication and permission mechanisms.

The application will not store plaintext passwords in Firestore or expose sensitive authentication credentials in the frontend code.

Gemini Image and Audio Processing

PlatePal will integrate the Google Gemini API to perform multimodal analysis of food photographs and recorded audio descriptions. The application will use Gemini's image-understanding capabilities to identify visible food items, estimate portion sizes, and generate approximate nutritional information.

Users will select photographs through an authenticated external photo service. The React frontend will send the authorized meal image to the Node.js/Express backend, which will securely communicate with the Gemini API.

The application will also allow users to record spoken meal descriptions using their device's microphone. These recordings will provide additional context about ingredients, portion sizes, cooking methods, and other details that may not be apparent from the photograph. The recorded audio will be submitted to the backend for processing by a Gemini model that supports audio input.

Gemini will analyze the available image and audio information and return structured results containing identified food items, estimated calories, approximate macronutrient values, and relevant observations. If information is insufficient, the application may ask users to clarify ingredients or portions before finalizing the analysis.

The resulting information will be displayed through PlatePal's existing React interface, replacing the hardcoded nutritional examples used in Project 1. Nutritional values will be labeled as estimates because image and audio analysis cannot guarantee precise nutritional measurements.

External Photo Service Integration

PlatePal proposes using the Google Photos Picker API to allow users to select meal photographs from their Google Photos libraries. Users will authorize access through Google OAuth 2.0 and explicitly select the photographs they wish to share with PlatePal. The selected images will be retrieved through authorized API requests and submitted for analysis using Google Gemini. This integration will provide user-controlled access to external photo content without requiring users to grant unrestricted access to their photo libraries.

Cross-Platform Compatibility

PlatePal will be designed as a responsive web application accessible from Windows, macOS, iOS, and Android devices through supported web browsers. In addition to authenticated Google Photos integration, users will be able to select meal photographs directly from their device's photo library or file system. This approach allows users who store photographs in Apple Photos, iCloud, or other local storage locations to use PlatePal without requiring a Google Photos account. Local photo selection will supplement, rather than replace, the authenticated external photo-service integration required for the project.

Google ADK Agent Architecture

PlatePal will use Google's Agent Development Kit (ADK) to implement a Meal Analysis Agent that coordinates the application's AI-powered food analysis and recommendation process. Rather than relying on a single Gemini response, the agent will manage a structured workflow that combines information from food photographs, recorded audio descriptions, and user preferences.

The proposed agent architecture will include three functional stages:

Food Identification: The agent will invoke Gemini to examine the selected meal photograph and identify visible food items, ingredients, and approximate portion sizes.

Audio Context Processing: The agent will use Gemini to interpret the user's recorded meal description, identifying additional details about cooking methods, ingredients, or portion sizes. This information will supplement photographic analysis.

Personalized Recommendations: The agent will combine the image and audio findings, account for the user's selected companion style (Casual, Strict, or Athlete), and produce estimated nutritional information and relevant meal suggestions.

The Meal Analysis Agent will communicate with the Node.js/Express backend through an authenticated service interface. The backend will validate requests, coordinate access to the agent, and manage storage of completed results in Google Firestore.

This architecture provides a meaningful role for Google ADK by allowing the application to coordinate multiple analysis steps, incorporate user-provided context, and produce structured results tailored to individual preferences.

Firestore Data Storage

PlatePal will use Google Cloud Firestore, a NoSQL document database, to store and retrieve user profiles, meal analysis records, and application preferences.

The proposed database will contain two primary collections, Users and Meals, which will also support application's meal analysis history feature.

Users: Stores each user's unique authentication identifier, display name, email address, selected companion style (Casual, Strict, or Athlete), and application preferences. User passwords will be managed by Firebase Authentication and will not be stored in Firestore.

Meals: Stores meal analysis records associated with authenticated users, including the analysis date, identified food items, estimated portion sizes, calories, macronutrients, and AI-generated recommendations. Each record will be associated with the user's unique authentication identifier.

Analysis History: The application will retrieve previously saved meal records to display a user's analysis history. This history can be generated by querying the Meals collection rather than maintaining a separate collection containing duplicate data.

Meal photographs may be stored separately in cloud object storage, when necessary, with Firestore maintaining authorized references or metadata rather than storing raw image files.

The Node.js/Express backend will manage authorized database operations, including creating meal records, retrieving saved analyses, updating user preferences, and deleting

records when requested. Firebase authentication tokens will be verified before protected database operations, and access will be limited to the authenticated user's information.

Firestore will enable PlatePal to provide persistent meal histories and personalized experiences across user sessions.

React Frontend Reuse and Enhancement

Individual Component Descriptions

Reusing the MealPicker Component

The existing MealPicker.jsx component from Project 1 will be reused and expanded in Project 2. Currently, the component allows users to select a food photograph from their local device or choose a sample meal image.

For Project 2, the component will be enhanced to support authenticated photo selection from an approved external photo service. Selected images will be passed to the application's backend for processing through the Google Gemini API and Google ADK Meal Analysis Agent. The existing image-selection functionality will be retained for development and testing. A separate audio recording component will also be introduced so users can provide spoken descriptions of their meals.

Reusing the MealPreview Component

The existing MealPreview.jsx component will be retained and enhanced for Project 2. In Project 1, this component displays a default illustration, a sample meal image, or a photograph selected by the user.

For Project 2, the component will also support displaying images retrieved from an authenticated external photo service. This allows users to preview their selected meal before submitting it for AI analysis. The existing image display and caption functionality will

be reused, reducing unnecessary frontend development while maintaining a consistent user interface.

Reusing the SampleResults Component

The existing SampleResults.jsx component from Project 1 will be modified to display dynamic AI-generated meal analysis results. Currently, the component uses predefined food items, calorie values, and recommendations that remain unchanged regardless of the selected photograph.

For Project 2, these hardcoded values will be replaced with structured analysis results returned by the Node.js/Express backend after processing through Google Gemini and the ADK Meal Analysis Agent. The updated component will display identified food items, estimated portion sizes, approximate calorie values, and personalized meal recommendations. The existing results layout, conditional rendering, and accessibility features will be retained where appropriate. This approach allows the group to reuse Project 1's frontend while extending its functionality to support real AI-powered food analysis.

Reusing the LoginForm Component

The existing LoginForm.jsx component from Project 1 will be retained and integrated with a secure authentication service in Project 2. The current component provides username and password fields, a sign-in button, and an option to create a new account.

For Project 2, the group proposes using Firebase Authentication to securely manage user accounts and login sessions. The React frontend will submit authentication requests through the selected authentication service, while the Node.js/Express backend will validate authentication tokens before allowing access to protected application resources. Users will also be able to authorize access to an approved external photo service. This

authorization will be handled separately from PlatePal account authentication. Reusing the existing login component will maintain a consistent interface while extending the application to support authenticated user sessions and secure access to saved meal-analysis results.

Reusing the CreateAccountForm Component

The existing CreateAccountForm.jsx component will be reused and integrated with a secure account registration system. In Project 1, the component collects a user's name, email address, username, and password using React state.

For Project 2, the registration interface will be connected to Firebase Authentication to support persistent user accounts. The application will validate registration information, securely manage credentials, and associate authenticated users with their meal histories and saved analysis results in Google Firestore. The existing form layout and reusable FormField.jsx component will be retained, while the account creation process will be enhanced to support secure authentication and appropriate input validation.

Proposed New React Components

In addition to reusing existing React components from Project 1, PlatePal will introduce several new components to support its expanded functionality.

PhotoImporter.jsx: This component will allow users to authorize access to an approved external photo service and select photographs for meal analysis. It will work alongside the existing MealPicker component.

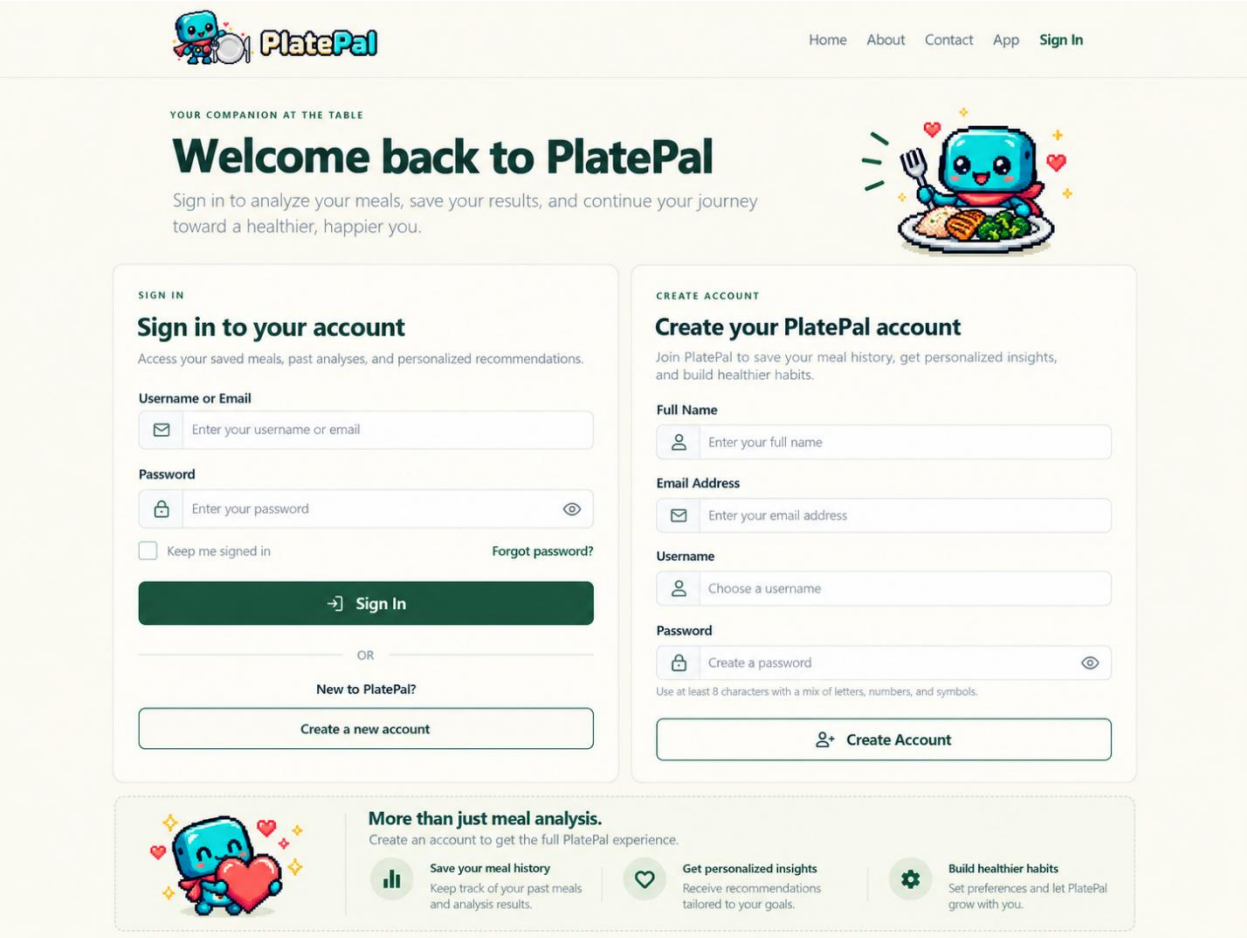
AudioRecorder.jsx: This component will allow users to record short spoken meal descriptions using their device's microphone. The recorded audio will be submitted for Gemini processing to provide additional information about ingredients, cooking methods, and portion sizes.

MealHistory.jsx: This component will display previously saved meal analyses retrieved from Google Firestore through the Express backend. Users will be able to review past results and select individual meals for additional details.

AnalysisStatus.jsx: This component will display processing indicators, progress messages, and error notifications while the application communicates with the backend and AI services. The existing SampleResults component will also be updated to display structured Gemini analysis results instead of predefined sample data. Together, these components will expand the existing PlatePal interface while preserving the original application's navigation, branding, styling, and reusable React structure.

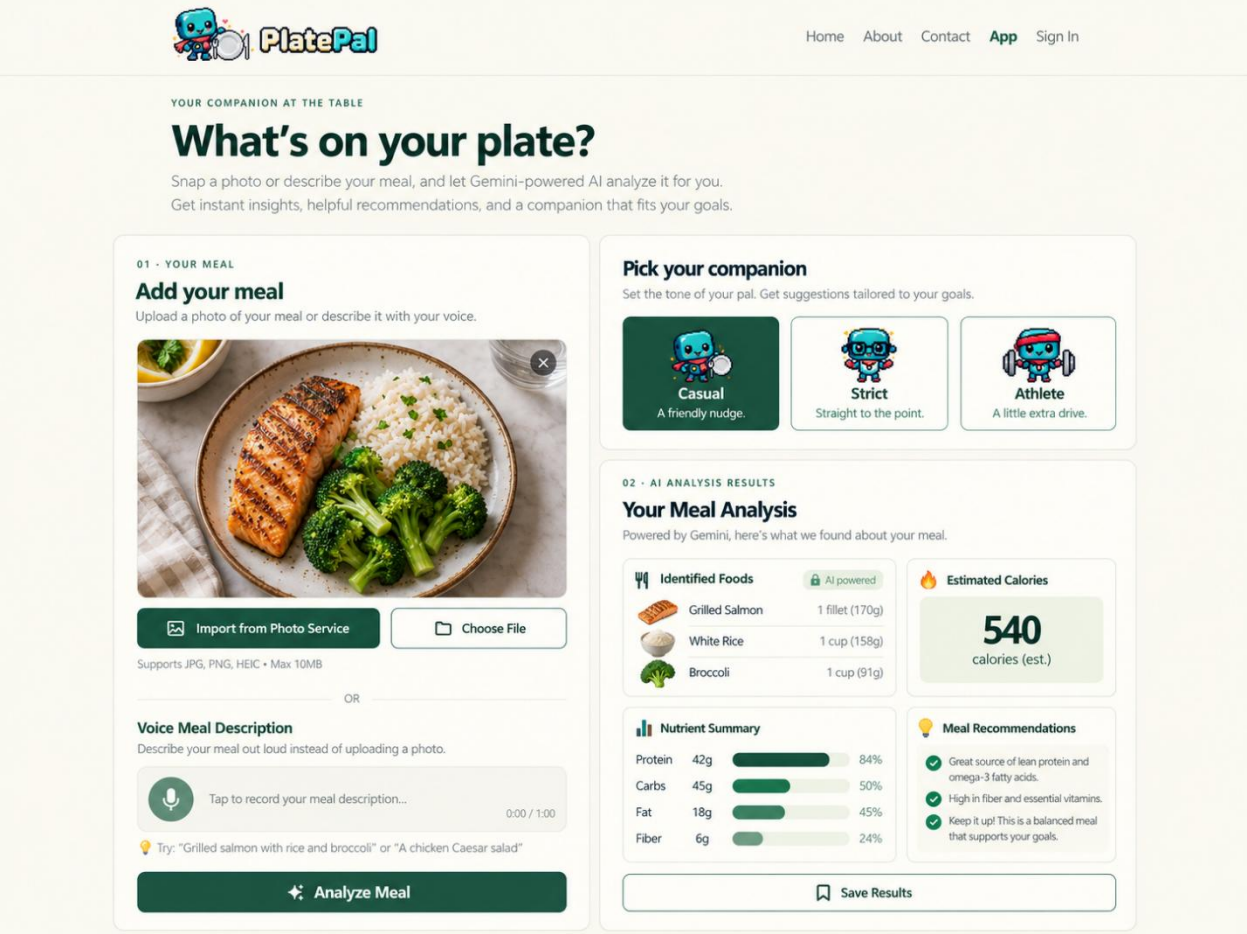
2. Proposed User Interface Mockups

Figure 1: User Authentication and Account Creation



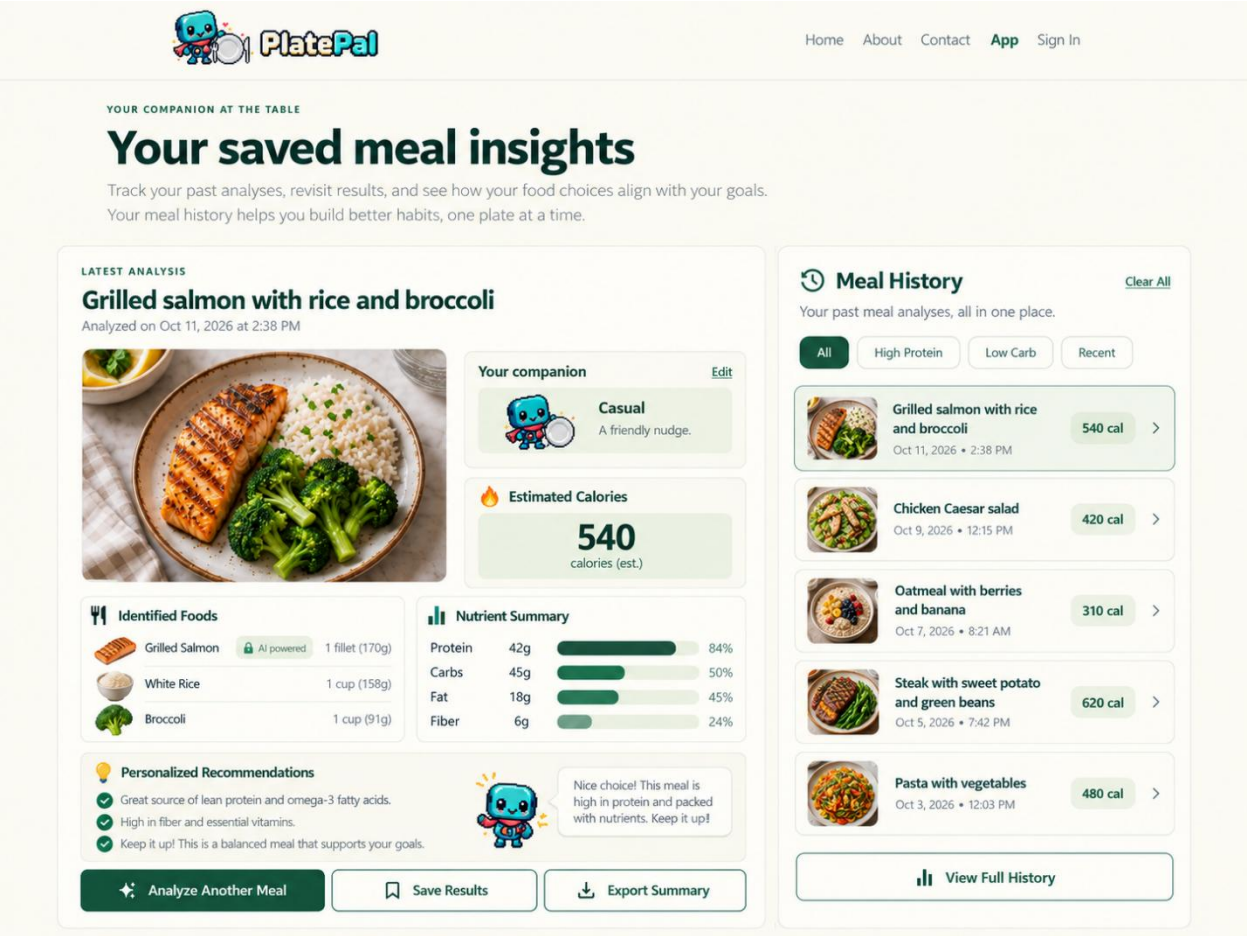
This proposed interface builds upon the existing login and account creation components from Project 1. Users will be able to securely sign in, create accounts, and access personalized meal histories and saved analysis results through Firebase Authentication.

Figure 2: AI-Powered Meal Analysis Interface



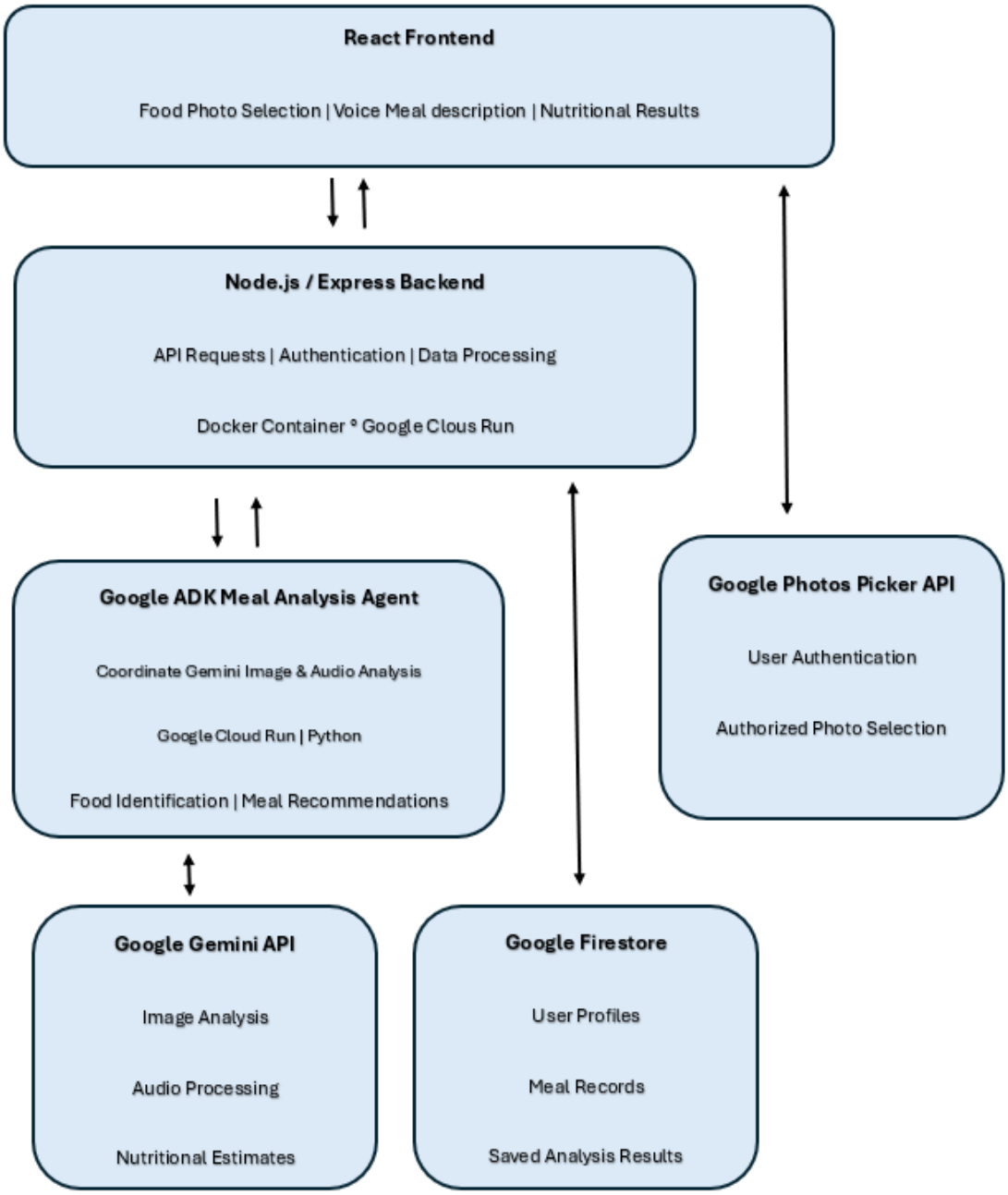
The proposed meal analysis interface expands the existing PlatePal application by introducing authenticated photo-service integration, voice meal descriptions, and Google Gemini-powered analysis. Users can select their preferred companion style (Casual, Strict, or Athlete), submit meal information, and receive estimated nutritional information and personalized recommendations.

Figure 3: Meal Analysis and History



This proposed interface allows authenticated users to review previously analyzed meals, including identified food items, estimated calories, nutrient summaries, and AI-generated recommendations. Google Firestore will store meal analysis records, allowing users to retrieve past results and track their meal histories over time.

3. Proposed System Architecture



Backend Infrastructure and Cloud Deployment

PlatePal will use a Node.js and Express backend to manage communication between the React frontend, Google Gemini API, Google Agent Development Kit (ADK), and Google Firestore database. The backend will receive requests from the frontend, coordinate food image and audio analysis, and return structured results to the user. Firestore will store user profiles, meal histories, and saved nutritional estimates.

For deployment, the application backend will be packaged in a Docker container and hosted using Google Cloud Run. This approach provides a consistent runtime environment and allows the backend to scale based on incoming requests.

API credentials and sensitive configuration values will be managed through secure environment variables or Google Secret Manager rather than stored in the GitHub repository. The proposed deployment architecture will allow frontend and backend development to proceed independently while supporting integration testing before the final release.

Google ADK Agent Deployment

The proposed Google ADK Meal Analysis Agent will be implemented as a separate Python-based service deployed on Google Cloud Run. The Node.js/Express backend will communicate with the agent through authenticated HTTPS requests. The agent will coordinate Gemini image and audio analysis and return structured results to the backend. Separating the application backend and agent service will allow each component to be developed, tested, and deployed independently while maintaining secure communication between services.

4. Technical Flow Control

1. **User Authentication:** The user accesses PlatePal through the React frontend and authenticates with the application. The user can then authorize access to an approved external photo service.
2. **Food Photo Selection:** The user selects a food photograph from the connected photo service. The application retrieves the authorized image for analysis.
3. **Voice Meal Description:** The user may record a short voice description providing additional information about the meal, such as ingredients, portion sizes, or preparation methods.
4. **Backend Processing:** The React frontend sends the meal information to the Node.js/Express backend through HTTPS requests. The backend validates the request and initiates the analysis process.
5. **AI Agent Coordination:** The Google ADK Meal Analysis Agent coordinates the processing of the food photograph and recorded audio using the Google Gemini API.
6. **Gemini Analysis:** Gemini identifies visible food items, interprets the spoken description, and generates estimated nutritional information based on the available details.
7. **Database Storage:** The Express backend saves relevant user information, meal history, and completed analysis results in Google Firestore.
8. **Results Display:** The backend returns the analysis results to the React frontend, where the user can view estimated nutritional information and suggested meal improvements.
9. **Cloud Deployment:** The backend application will be containerized using Docker and deployed through Google Cloud Run, providing a managed environment for handling application requests.